Domain-Expert AI Collaboration: Crossing the Paradigm Boundary from Industrial Control to Application Software

A Case for Engineering Judgement as the Scarce Resource in AI-Assisted Development

Stuart J. Hunt

Independent Researcher; Automation Engineer & Process Control Specialist (ISA S88/S95/S106)

Correspondence: stuartj.hunt@sky.com · GitHub: github.com/stuartj1-1981/Quantum-Swarm

Preprint deposited April 2026 · CC BY 4.0 · doi:10.5281/zenodo.19382919

Abstract

The dominant narrative around AI-assisted software development assumes a professional software engineer directing an AI coding agent. The human supplies architectural knowledge, code review capability, and software craftsmanship; the AI accelerates implementation. This framing excludes a second, undocumented model: a domain expert whose programming experience lies in a different paradigm — industrial control languages rather than application software — using AI agents to cross the paradigm boundary and build production systems that would otherwise require a software development team.

This paper presents evidence from a 14-week production deployment of Quantum Swarm Heating (QSH), a real-time energy transfer control system comprising 22 deterministic controllers, a digital twin simulation engine, a system identification module that learns physical parameters from passive observation, a React/TypeScript web interface, and an InfluxDB historian. The system architecture is domain-agnostic — the pipeline manages energy flows between sources and zones based on learned physical parameters and configurable control strategies. Its current deployment is residential heat pump optimisation across three live UK installations. The system was built entirely through structured human–AI collaboration under a documented governance framework (Hunt, 2026b), without employing or contracting software developers.

The author — a process control specialist with 28 years of industrial automation experience, fluent in IEC 61131-3 languages (LAD, FBD, STL, CFC, SFC, SCL), C (ANSI), and VBScript, but with no prior experience building application software stacks — supplied the control architecture, the physics models, the engineering constraints, and the process discipline. AI agents (Anthropic Claude, multiple model tiers) supplied the application software implementation capability: Python, TypeScript, React, FastAPI, WebSocket, database integration, and frontend rendering.

This is not vibe coding. The term, coined by Karpathy (2025), describes unstructured prompting where the human accepts AI output without inspection. The methodology documented here is its structural inverse: every change governed by a numbered instruction document, assessed by an independent AI agent without codebase access, implemented by a third agent, and validated by a fourth. The human retains engineering authority at every stage. 64 instruction documents, 49 test files, and 348,170 fleet simulation runs provide the evidence base.

The paper argues that domain expertise — not application software skill — is the binding constraint in AI-assisted development of technically complex systems. The central thesis is one of **convergence**. Three elements, each individually precedented, converge in this case for the first time in documented literature: (1) the crossing of an industrial-to-application paradigm boundary by an experienced control systems programmer, (2) the production deployment of the resulting system on physical hardware governing real energy transfer processes, and (3) the transfer of GAMP 5 validation principles, FMEA methodology, and ISA-standard process discipline to govern the AI collaboration. Each element exists in isolation across the literature — spec-driven development addresses governance, citizen development addresses non-traditional developers, AI-assisted coding addresses productivity — but no published case as of April 2026 reports their

convergence in a single project with production evidence.

The implications extend to any domain where this convergence is achievable: where deep specialist knowledge and programming competence exist in a paradigm that the application software industry does not recognise, and where regulated-industry process discipline can govern the AI collaboration. Building services engineering, process control, clinical systems, and scientific instrumentation are immediate candidates.

This work did not begin as a deliberate experiment in human–AI collaboration or governance frameworks. It originated in early 2025 from a practical need to improve underperforming residential heat pump control in the author's own property. The structured collaboration model, air-gapped pipeline, and convergence thesis emerged iteratively as the only viable way to cross from industrial control paradigms into a production-grade application software stack capable of governing physical hardware in occupied homes.

1. Introduction

AI-assisted software development has moved from experimental to mainstream. McKinsey's 2025 State of AI report found 78% of organisations using AI in at least one business function. GitHub Copilot, Anthropic Claude Code, and comparable tools are now standard in professional software teams. The conversation has advanced from "can AI write code?" to "how do we govern AI-written code?" — a question addressed by spec-driven development (Thoughtworks Technology Radar, 2025), GitHub's spec-kit framework, and the author's own air-gapped pipeline (Hunt, 2026b).

All of these frameworks share an unstated assumption: the human in the loop is a software engineer. The spec-driven development literature describes developers writing specifications for AI agents. The agentic engineering discourse (Karpathy, 2026) discusses engineers designing systems where AI agents plan, write, test, and ship code. The CLAUDE.md convention (Anthropic, 2025) provides project-level instructions from developers to AI coding tools.

This paper challenges that assumption. It presents a documented case where the human contributor is not a software engineer but a process control specialist — with 28 years of experience in industrial automation (ISA S88 batch control, ISA S95 operations management, ISA S106 procedure automation), fluent in IEC 61131-3 programming languages, C, and VBScript, but with no prior experience in the application software paradigm: no Python backend, no web frontend, no REST API, no database integration, no JavaScript ecosystem.

The system built through this collaboration is not a prototype, a demo, or a proof of concept. It is a production real-time control system governing physical hardware — heat pumps, thermostatic radiator valves, flow temperatures — in occupied residential buildings across three live installations, executing a 22-controller pipeline every 30 seconds, learning building thermal parameters from passive observation, and serving a web interface with live telemetry. It has accumulated over 200,000 control cycles across the fleet. The development was governed by a GAMP 5-inspired four-stage pipeline (Hunt, 2026b) applying ISA-standard document control, independent assessment, and mechanical validation — process discipline transferred directly from pharmaceutical and industrial automation practice to AI-assisted software development.

The project was not conceived as a case study in AI-assisted development. It began as targeted remediation of unsatisfactory heat pump behaviour observed in the author's own property. Over 14 weeks, the combination of domain-driven control architecture, physics-based system identification, and the practical necessity of building a modern web-enabled interface and historian led to the adoption — and subsequent formalisation — of a GAMP-inspired governance process. The convergence of paradigm crossing, production physical deployment, and regulated-industry governance transfer was not designed top-down. It emerged from solving a real engineering problem under real constraints. This organic origin is central to the transferability claim: if the model arose from practical necessity rather than theoretical design, other domain experts facing similar frustrations can replicate the pattern without needing to start with a grand theory.

1.1 Contributions

This paper makes four contributions:

1. It documents the first published case of an industrial control programmer crossing the paradigm boundary to application software through structured AI collaboration, building a production real-time control system with quantified evidence of system complexity, test coverage, and operational duration.

2. It distinguishes domain-expert AI collaboration from vibe coding on structural grounds — governance framework, specification quality, engineering authority retention — rather than on subjective quality judgement.

3. It identifies domain expertise — including the programming intuition and algorithmic thinking that industrial control work develops — as the binding constraint in AI-assisted development of technically complex systems, arguing that application software implementation capability is now supplantable by AI agents when governed by adequate process discipline.

4. It proposes a transferability thesis: the model applies to any domain where (a) deep specialist knowledge constrains system design, (b) the domain expert can specify requirements with engineering precision, and (c) a governance framework enforces specification-implementation fidelity.

2. Background and Related Work

2.1 Vibe Coding and Its Discontents

Andrej Karpathy introduced the term “vibe coding” in February 2025, defining it as “fully giving in to the vibes, embracing exponential curves, and forgetting that the code even exists.” The approach generated significant interest and equally significant criticism. By early 2026, industry commentary had converged on a distinction between vibe coding (unstructured, uninspected, disposable) and AI-assisted engineering (structured, reviewed, production-grade) (Osmani, 2026; TATEEDA, 2026).

This distinction, while useful, remains anchored in the assumption that the human participant is a software engineer. Osmani’s workflow assumes familiarity with code review, architectural patterns, and test design. The “vibe coding hangover” literature (Context Studios, 2026) warns that domain expertise, systems thinking, and security awareness remain essential — but frames these as software engineering competencies, not domain-specialist competencies.

2.2 Spec-Driven Development

Spec-driven development (SDD) emerged as the structured alternative to vibe coding. The core principle is separation of planning from execution: the human writes a detailed specification; the AI agent implements it; the human reviews the output against the specification (Thoughtworks, 2025; GitHub spec-kit, 2025; JetBrains, 2025).

SDD represents Stage 1 of the governance model used in this work. The air-gapped pipeline (Hunt, 2026b) extends SDD by adding independent assessment (Stage 2), controlled implementation (Stage 3), and mechanical validation (Stage 4). The key innovation is structural separation: no single agent participates in more than one stage, preventing the systematic blind spots that arise when the specifier, reviewer, implementer, and validator share context.

2.3 Domain Experts and AI-Assisted Development

The academic literature on domain expert involvement in AI systems focuses primarily on LLM evaluation and design (arxiv:2602.14357), not on domain experts as primary developers. Industry reports describe “citizen developers” using no-code and low-code platforms, but these platforms constrain the solution space to pre-built components. The C3 AI blog describes non-technical users building “composable building blocks” with autonomous coding agents, but does not document complex real-time systems.

A Loomery case study (2026) describes a non-engineer building applications with AI coding tools, noting productivity gains but also fundamental limitations: non-programmers using code generation tools solved an average of only 1.4 out of 4 programming problems. This finding is unsurprising and highlights a critical

distinction. The QSH case differs from the “citizen developer” model in a fundamental respect: the author is not a non-programmer. Twenty-eight years of programming in IEC 61131-3 languages, C, and VBScript develops algorithmic thinking, debugging intuition, state machine design, and the ability to reason about program flow — competencies that transfer across paradigms even when the specific languages and frameworks do not. The limiting factor is not programming ability per se, but the combination of domain knowledge, programming intuition, and specification discipline that industrial control experience develops.

A systematic survey of public literature as of April 2026 — spanning academic databases, preprint repositories, industry blogs, and conference proceedings — found no documented case combining three specific elements: (1) a programmer from an industrial control paradigm (IEC 61131-3, C, or equivalent) crossing to application software through AI collaboration, (2) production deployment of the resulting system on physical hardware, and (3) governance of the collaboration through a formal framework derived from regulated industry practice (GAMP, ISA, or equivalent). Each element exists in isolation: spec-driven development addresses governance, citizen development addresses non-traditional developers, and AI-assisted coding addresses productivity. The intersection — where all three converge in a single documented project with production evidence — appears to be unoccupied.

2.4 Industrial Process Control as Source Domain

The methodology documented here draws directly from industrial process control practice. Three ISA standards and one analytical method are particularly relevant:

- **ISA-88 (Batch Control):** Separates recipe (what to do) from equipment control (how to do it). The instruction document in this methodology serves the same function as the master recipe in S88: it captures intent and sequencing, leaving implementation details to the execution layer.
- **ISA-95 (Enterprise-Control Integration):** Defines clear boundaries between business, manufacturing operations, and control levels. The four-stage pipeline enforces analogous boundaries between specification, assessment, implementation, and validation.
- **GAMP 5 (Pharmaceutical Validation):** Provides the qualification framework (IQ, OQ, PQ) mapped to pipeline stages in Hunt (2026b). The principle that the entity performing work cannot also verify it is foundational to GxP practice and directly motivates the air-gap architecture.
- **FMEA (Failure Mode and Effects Analysis, IEC 60812):** A systematic method for identifying potential failure modes, their effects, and mitigations before a system is built. In industrial automation, FMEA is performed during design to ensure that every controller, interlock, and operating mode accounts for foreseeable failures. Applied to AI-assisted software development, FMEA produces the engineering constraints and edge-case specifications that prevent AI agents from generating technically correct but operationally unsafe implementations. This is a fundamentally different design methodology from the test-driven or feature-driven approaches that dominate application software development, and its absence from the AI-assisted development literature is a significant gap.

3. The QSH System: Scope and Complexity

Quantum Swarm Heating (QSH) is a real-time energy transfer control system. Its architecture — a driver-agnostic controller pipeline, a system identification engine that learns physical parameters from passive observation, and a digital twin for offline simulation — is general-purpose by design. The pipeline does not know it is controlling a heat pump; it manages energy flows between sources and zones based on thermal state, learned parameters, and configurable control strategies. The current deployment is residential heat pump optimisation, delivered as a Home Assistant add-on across three live UK installations. A full technical description of the residential application appears in Hunt (2026a). This section summarises the system’s scope to establish the complexity of the software built through domain-expert AI collaboration.

3.1 Control Pipeline

The core pipeline comprises 22 deterministic controllers executing in sequence every 30 seconds via a shared CycleContext. Controllers include: thermal state estimation, weather compensation, cascade PI control, flow temperature management, shoulder mode (demand-gated operation), antifrost override, summer shutdown, TRV setpoint management, and historian recording. The pipeline is driver-agnostic: an IODriver protocol with a factory pattern supports any backend that implements the driver contract. Current drivers include a live Home Assistant API driver and a mock driver that wires the pipeline to the digital twin for closed-loop simulation. The separation of control logic from I/O hardware is a direct application of ISA-88's procedural/equipment split — the same pipeline could govern a different energy transfer domain with a different driver and configuration set.

3.2 System Identification

A system identification module learns per-zone physical parameters from passive observation. In the current residential deployment, these parameters characterise building thermal behaviour:

- **U (heat loss coefficient, kW/°C):** Learned from source-off cooling periods. Converges within 10–50 observations per zone.
- **C (thermal mass, kWh/°C):** Learned primarily from passive cooling decay analysis (Newton's law of cooling fit, $R^2 > 0.8$ gate). Per-cycle estimation is structurally constrained in multi-zone installations where per-zone energy input rarely exceeds energy loss.
- **Solar gain factor:** Learned from sunny source-off periods with mature U estimates.

The system identification approach is not specific to building physics. Any energy transfer system where zones gain and lose energy at rates governed by identifiable physical parameters is amenable to the same methodology. All observation gates are instrumented for audit, with rejection reasons categorised and counted.

3.3 Digital Twin

A ThermalEngine provides per-zone energy balance simulation with configurable physical parameters, emitter models, source efficiency curves, environmental gain, infiltration, and part-load ratio effects. For the residential deployment, nine UK residential archetypes are defined in YAML profiles with a target of 84, and ten UK climate zone weather files provide hourly environmental data. The twin enables closed-loop simulation of the full control pipeline without physical hardware — equivalent to a factory acceptance test (FAT) environment in industrial automation.

3.4 Web Interface

A React 19 / TypeScript 5.9 frontend (Vite 8, Tailwind CSS 4) provides real-time monitoring via WebSocket, zone-by-zone status, comfort control, schedule management, away mode with per-zone recovery tracking, engineering diagnostics, and a SCADA-style historian trend viewer with dual Y-axis, multi-trace display, and CSV export. The backend is FastAPI on port 9100 with REST endpoints for configuration, control, system identification data, and InfluxDB historian queries.

3.5 Fleet Simulation

348,170 parameterised simulation runs across 8 heat pump models, 8 UK archetypes, 10 climate zones, 27 weather compensation curves, and 8 control strategies. Results are stored in a 2 GB SQLite database and form the evidence base for the residential deployment analysis (Hunt, 2026a).

3.6 Current Deployment: Residential Heating

The current deployment targets residential heat pump optimisation in UK housing. Three installations are operational:

- **Installation 1:** 205 m^2, 13 zones, 2016 detached, Octopus Cosy 6 heat pump. 100,000+ cycles. U converging well across all zones. C via passive cooling (96 fits). Solar learning active.
- **Installation 2:** 10 zones. 100,000+ cycles. U strong (up to 375 observations per zone). No solar sensor. C via passive cooling (62 fits).
- **Installation 3:** In commissioning.

3.7 Test Coverage

49 test files across pipeline and twin modules (Python pytest). Frontend test suite covering hooks, utilities, and components (Vitest, Testing Library). Integration tests cover full controller chain, state persistence, and orchestration.

4. The Collaboration Model

4.1 What the Human Contributed

The author's contribution spans engineering judgement, algorithmic design, and process discipline — drawing on competencies developed across 28 years of industrial automation programming:

- **Control architecture:** Pipeline structure, controller sequencing, state machine design, seasonal mode transitions (winter equilibrium, shoulder demand-gating, summer shutdown). These are direct transfers from IEC 61131-3 SFC (Sequential Function Chart) design patterns.
- **Physics models:** Heat balance equations, emitter characteristics, COP modelling, system identification algorithms, passive cooling analysis methodology.
- **Algorithmic design:** EMA convergence strategy, observation gating logic, confidence-weighted learning rates, passive cooling curve fitting methodology. The author specified these algorithms in detail; the AI agents implemented them in Python.
- **Failure mode analysis:** Every controller and operating mode was designed through FMEA (Failure Mode and Effects Analysis) before implementation. Shadow mode safety (no hardware writes when control disabled), antifrost override logic, compressor protection (minimum off-time, restart dwell), TRV actuator type handling — each traces to a specific failure mode with identified effects and mitigations. This is standard practice in industrial automation (IEC 60812) and pharmaceutical validation, but largely absent from application software development.
- **Process discipline:** Instruction document convention, quality gate definitions, CLAUDE.md project rules, dependency tracking, lifecycle management.
- **Acceptance criteria:** What "correct" looks like for each controller, each mode, each edge case. The domain knowledge to distinguish a real bug from legitimate physics.

4.2 What the AI Contributed

AI agents (Anthropic Claude, models including Opus 4.6 and Sonnet 4.6) contributed implementation capability:

- **Languages:** Python 3.11, TypeScript 5.9, React 19, SQL, YAML, CSS, Bash.
- **Frameworks:** FastAPI, WebSocket, Recharts, Tailwind CSS, Vite, Vitest, pytest.
- **Patterns:** Driver abstraction (IODriver protocol), factory methods, EMA convergence, ring buffers, thread-safe singletons, REST/WebSocket API design, responsive layout.
- **Infrastructure:** Home Assistant add-on packaging, InfluxDB integration, Docker configuration, CI pipeline, ESLint/TypeScript strict mode compliance.

4.3 What Neither Could Do Alone

The AI agents could not have designed the QSH control architecture. They lack the 28 years of process control experience that informed the pipeline structure, the seasonal mode design, the system identification approach, and the physics-grounded acceptance criteria. When an AI agent was given architectural latitude (Section 5.2), it produced work that was technically correct but violated the project's engineering conventions — demonstrating competent implementation without engineering judgement.

The author could not have crossed the paradigm boundary in 14 weeks without AI agents. The application software stack — Python backend patterns, TypeScript/React frontend, WebSocket real-time data, InfluxDB time-series storage, Vite build tooling, Tailwind CSS, Testing Library, pytest — represents a different ecosystem from IEC 61131-3/C/VBS development. The programming fundamentals transfer (loops, conditionals, state machines, debugging); the frameworks, idioms, toolchains, and ecosystem conventions do not. AI agents bridged this gap, translating the author's algorithmic specifications into idiomatic application software.

The collaboration is multiplicative, not additive. Neither party's contribution is reducible to a fraction of the other's. Domain expertise and programming intuition without application software fluency produces specifications that cannot be self-implemented. Application software fluency without domain expertise produces vibe-coded prototypes that cannot govern a physical system.

5. Why This Is Not Vibe Coding

5.1 Structural Differences

The term "vibe coding" has acquired negative connotations in the software engineering community, and for good reason: it describes a process with no quality assurance, no specification, no review, and no accountability. The methodology used to build QSH differs from vibe coding on every structural dimension:

Dimension	Vibe Coding	QSH Methodology
Specification	Natural language prompt, ephemeral	Numbered instruction document, version-controlled
Review	None, or same-session self-review	Independent air-gapped assessment (Stage 2)
Implementation	Single agent, single session	Dedicated implementation agent (Stage 3)
Validation	None	Context-free mechanical diff check (Stage 4)
Human role	Accept or reject output	Engineering authority at every stage
Code inspection	"Forget that the code even exists"	TypeScript strict, ESLint zero-warning, pytest
Governance	None	64 numbered instructions, dependency chains
Test coverage	Incidental	49 test files, mandatory for new components
Physical system	Not applicable	Three live installations, real-time control

5.2 Evidence from Process Violations

On 1 April 2026, an audit was conducted where one AI model (Opus 4.6) reviewed instruction documents produced by a different AI model (Sonnet 4.6). The audit found that all code changes were technically correct — the implementations worked, the bugs were real, the fixes were sound. However, every instruction document violated the project's established process conventions: no sequential numbering, no standard header blocks, no lifecycle management (completed work left in the active directory).

This audit demonstrates the distinction operationally. A vibe-coded project would have no process to violate. QSH has a process sufficiently rigorous that an AI agent's failure to follow it is detectable, classifiable (five severity levels), and correctable. The code quality was adequate; the engineering discipline was not. The human's role is precisely to maintain that discipline.

5.3 The Engineering Judgement Test

The clearest distinction between vibe coding and domain-expert AI collaboration is the nature of the human's contribution. In vibe coding, the human says "build me an app that does X." In domain-expert collaboration, the human says:

> *"The passive cooling analyser must fit Newton's law of cooling to extended HP-off windows, extract tau = C/U from the decay curve, gate on $R^2 > 0.8$, and derive C = tau × U_effective only when U is mature (u_observations ≥ 10). The R^2 from the fit weights the EMA learning rate: higher-confidence fits pull harder."*

This is not a vibe. It is an engineering specification grounded in thermodynamic first principles, instrumented for observability, and gated for statistical confidence. The AI agent implements it; the human designed it.

Behind each specification sits an FMEA process that most software developers would not recognise. Before the passive cooling analyser was specified, the author had already identified: what happens if the R^2 gate is too loose (noisy C estimates propagate through the model), what happens if it is too tight (insufficient observations, C never converges), what happens if U is immature when tau is extracted (circular dependency, biased C), and what the effect of each failure mode is on downstream controllers. The specification is not a feature request — it is the output of a structured failure analysis. This is how industrial automation systems are designed, and it is the methodology that produces specifications precise enough for AI agents to implement without ambiguity.

6. The Binding Constraint Argument

6.1 Software Skill Is Now Supplantable

The traditional model of software development assumes that implementation capability — the ability to write correct, maintainable, tested code in appropriate languages and frameworks — is a scarce human resource requiring years of training and experience. This assumption underpins university computer science programmes, software engineering hiring practices, and the economic structure of the technology industry.

AI coding agents have made application software capability abundant. Not perfect, not unsupervised, not ungoverned — but available on demand at a level sufficient for production deployment when adequately specified and verified. The evidence from QSH is that a 22-controller real-time system with a React frontend, an InfluxDB historian, a digital twin, and 49 test files can be built to production quality by a domain expert whose programming experience is in a different paradigm, provided:

1. The human possesses deep domain expertise in the problem being solved.
2. A governance framework enforces specification quality and implementation fidelity.
3. The human retains engineering authority over architectural and physics decisions.

6.2 Domain Expertise Is Not Supplantable

The inverse does not hold. AI agents cannot supply the 28 years of process control experience that informed QSH's control architecture. They do not know that UK multi-zone heat pumps spread 5–8 kW thermal across 10–13 zones, making per-room net heat input structurally insufficient to exceed heat loss in most cycles. They do not know that the C-gate rejection rate of 70–88% is legitimate physics, not a bug. They do not know that shadow mode must suppress all hardware writes including side-channel shoulder commands — a constraint derived from operational experience with real TRV actuator behaviour.

These are not facts that can be retrieved from training data. They are engineering judgements formed through decades of observing physical systems, understanding failure modes, and developing intuition for what "correct" looks like in a control context. This is the scarce resource. This is the binding constraint.

6.3 Implications

If domain expertise is the binding constraint and software implementation is supplantable, then the limiting factor in building technically complex software is not the availability of programmers but the availability of domain experts who can specify systems with engineering precision. This inverts the traditional hiring model for technical software projects and has implications for:

- **Building services engineering:** The UK heating industry lacks software development capacity but has deep domain knowledge in thermal systems, heat loss calculations, and installation practice.
- **Process control:** Industrial automation engineers understand control theory, safety systems, and regulatory requirements, and programme extensively in IEC 61131-3 languages, but are typically confined to vendor-provided platforms (DCS, PLC, SCADA). QSH is itself evidence that the paradigm boundary between control system programming and application software is crossable with AI assistance.
- **Clinical and scientific instrumentation:** Domain experts (clinicians, researchers, metrologists) understand the measurement problem but depend on software teams for data acquisition and analysis systems.
- **Agricultural technology:** Agronomists and soil scientists understand crop physiology and environmental interactions but lack the software infrastructure to deploy precision monitoring and control at scale.

In each case, the domain knowledge exists. The governance framework exists (Hunt, 2026b). The AI implementation capability exists. The missing piece is recognition that these three components are sufficient.

7. Transferability Conditions

The QSH experience suggests three necessary conditions for domain-expert AI collaboration to produce production-quality software:

7.1 Specification Precision

The domain expert must be able to specify requirements at a level of precision that eliminates ambiguity in implementation. This is not a universal skill among domain experts — it is a specific competence developed through practice in controlled environments. The author's background in ISA S88 recipe specification (where ambiguity causes batch failures) and FMEA practice (where unidentified failure modes cause incidents) directly transferred to writing instruction documents for AI agents. FMEA is particularly valuable because it forces the specifier to enumerate edge cases, failure modes, and boundary conditions before implementation begins — precisely the information that AI agents need but cannot generate from domain-naive reasoning.

Specification precision is trainable. The instruction document archive for QSH shows measurable improvement in specification quality over 64 documents, as assessed by the air-gapped review stage. Early instructions required multiple assessment iterations; later instructions typically passed assessment on the first cycle.

7.2 Governance Framework

A governance framework must enforce the boundary between specification (human) and implementation (AI). Without governance, the collaboration degrades toward vibe coding as the human cedes architectural decisions to the AI agent's defaults. The air-gapped pipeline (Hunt, 2026b) provides one such framework; others are possible provided they maintain structural separation between specification, review, implementation, and validation.

7.3 Domain Depth

The domain expertise must be sufficient to evaluate AI-generated implementations against physical reality. The human must be able to distinguish a correct implementation from one that compiles and passes tests but violates domain constraints. In the QSH context, this means recognising that a system identification algorithm might converge mathematically but produce physically implausible thermal parameters — a judgement that requires understanding building physics, not software testing.

8. Limitations and Scope

This paper documents a single case study. The following limitations apply:

1. **Single practitioner.** The author has 28 years of engineering experience including extensive programming in IEC 61131-3 languages, C, and VBScript, plus structured specification practice (ISA S88 recipes, GAMP validation protocols). The combination of programming fluency in one paradigm, domain expertise, and specification discipline may not be common among all domain experts. The transferability to domain experts without programming backgrounds or without specification discipline is not established.

2. **Single domain.** Building thermal control is well-suited to this model because the physics is deterministic, the control objectives are well-defined, and the failure modes are observable (room temperature). Domains with less observable outcomes or more ambiguous success criteria may be less amenable.

3. **Single AI provider.** All implementation was performed by Anthropic Claude models (Opus and Sonnet tiers). The methodology is agent-agnostic in principle, but the evidence base is specific to one provider's capabilities as of Q1 2026.

4. **No controlled comparison.** There is no parallel implementation by a traditional software team for direct comparison of development speed, defect rate, or maintenance burden. The claim is that the system was built and is operational, not that it was built optimally.

5. **Operational duration.** The system has been in continuous operation since January 2026 (14 weeks at time of writing). Long-term maintenance patterns, technical debt accumulation, and scalability under code growth are not yet characterised.

6. **Governance overhead.** The four-stage pipeline adds overhead per change. This is justified for architectural, physics, and integration changes but is acknowledged as disproportionate for trivial fixes. The QSH project's CLAUDE.md does not currently define a lightweight path for minor changes.

9. Discussion

9.1 The Gap Between Communities

Two communities are converging on the problem of AI-assisted development from opposite directions. The software engineering community brings expertise in code quality, testing, deployment, and tooling but lacks vocabulary for validation lifecycles, controlled documents, and the principle that the performer cannot verify their own work. The process automation community brings GAMP 5, ISA standards, and decades of controlled-document practice but has not yet engaged with AI agents as development tools.

Neither community has recognised the other's contribution. The spec-driven development literature reinvents controlled-document principles without citing GAMP or ISA. The process automation literature discusses AI for plant control without considering AI as a development methodology. This paper, and the two that precede it (Hunt, 2026a; Hunt, 2026b), sit in the intersection.

9.2 The Role of AI in Authorship

All three preprints in this series were developed through human–AI collaboration. The AI agents contributed implementation capability (code generation, formatting, literature search, structural suggestions). The author contributed all intellectual content: the control architecture, the physics models, the governance framework, the arguments, and the editorial decisions. No AI agent is listed as author. This is consistent with current academic norms (ICMJE, Zenodo) and with the thesis of this paper: AI supplies capability; the human supplies judgement.

9.3 Implications for Engineering Education

If domain expertise is the binding constraint and AI supplies application software capability, then the traditional assumption that domain engineers must retrain in application software languages to build custom systems is no longer valid. Engineers already learn to programme — IEC 61131-3 languages are taught and used throughout industrial automation — but the paradigm gap between control system programming and application software development has historically confined domain experts to vendor platforms (DCS, PLC, SCADA). AI agents dissolve that boundary. The implication for engineering education is not “learn to code” (they already do) but “learn to specify with the precision that AI agents require” — a competence that ISA S88 recipe development, GAMP validation protocols, and functional safety specifications already cultivate.

9.4 Origins and Emergence

It is worth noting that neither the collaboration model nor the governance framework was designed top-down. The author, frustrated with existing residential heat pump controls that applied fixed weather compensation curves without building-state feedback, set out to build something better using familiar process control principles. The requirement to integrate real-time telemetry, a user interface, persistent historian, and simulation capabilities forced a crossing of the industrial-to-application software boundary. The air-gapped pipeline and numbered instruction documents arose as necessary discipline to maintain engineering authority while leveraging AI implementation capacity. The FMEA methodology was not adopted for the purpose of AI governance — it was the author’s habitual approach to system design, applied reflexively because that is how industrial automation engineers work.

In that sense, the governance framework and collaboration model were discovered rather than invented. The convergence thesis articulated in this paper is a retrospective observation, not a prospective design. This distinction matters for transferability: the model does not require other domain experts to read this paper and adopt a methodology. It predicts that domain experts who already possess specification discipline and are forced by practical need to cross a paradigm boundary will independently arrive at similar structures — because the constraints that shaped this work are general, not specific to this author or this project.

10. Conclusions

This paper presents the first documented case of an industrial control programmer crossing the paradigm boundary to application software through structured AI collaboration, building a production real-time control system. The system — Quantum Swarm Heating — comprises 22 controllers, a digital twin, a system identification module, a web interface, an InfluxDB historian, and three live residential installations, built over 14 weeks under a GAMP-inspired governance framework.

The evidence supports three claims:

1. **Domain expertise — encompassing both specialist knowledge and the programming intuition that industrial control work develops — is the binding constraint** in AI-assisted development of technically complex systems. AI agents supply application software implementation capability sufficient for production deployment when governed by adequate process discipline.

2. **Domain-expert AI collaboration is structurally distinct from vibe coding.** The distinction is not subjective quality judgement but governance architecture: numbered specifications, independent assessment, controlled implementation, and mechanical validation.

3. **The model is transferable** to any domain where deep specialist knowledge constrains system design, the domain expert can specify requirements with engineering precision, and a governance framework enforces specification-implementation fidelity.

The central thesis is convergence. Paradigm crossing, production physical deployment, and regulated-industry governance transfer each exist independently in the literature. Their convergence in a single documented project — with 200,000+ control cycles, 64 instruction documents, and three live installations as evidence — is, to the author’s knowledge, without precedent as of April 2026. The three preprints in this series form a complete account of that convergence: the production deployment (Hunt, 2026a), the governance framework (Hunt, 2026b), and this paper documenting the paradigm crossing and the collaboration model that connects

them. The source code, instruction archive, fleet simulation database, and governance documentation are available under Apache 2.0 at github.com/stuartj1-1981/Quantum-Swarm and via Zenodo deposit.

11. Data Availability

QSH source code: github.com/stuartj1-1981/Quantum-Swarm (Apache 2.0)

Instruction archive: development/ and development/completed/ directories in the repository

Fleet simulation database: Zenodo deposit (Hunt, 2026a), ~2 GB SQLite

Governance framework: Zenodo deposit (Hunt, 2026b)

This preprint: Zenodo deposit, CC BY 4.0. doi:10.5281/zenodo.19382919

References

1. Hunt, S.J. (2026a). Physics-Based Thermal Optimisation for Residential Heat Pumps: Fleet Simulation, Weather Compensation Analysis, and the Shoulder Mode Discovery. Zenodo. doi:10.5281/zenodo.18903154
2. Hunt, S.J. (2026b). The Air-Gapped AI Development Pipeline: Applying GAMP-Inspired Validation Principles to Staged Agent Software Development. Zenodo. doi:10.5281/zenodo.19323404
3. Karpathy, A. (2025). Vibe coding. Twitter/X, February 2025.
4. Karpathy, A. (2026). Agentic engineering. Twitter/X, January 2026.
5. Osmani, A. (2026). Vibe coding is not the same as AI-assisted engineering. Medium.
6. Thoughtworks (2025). Spec-driven development. Technology Radar, Vol. 31.
7. GitHub (2025). spec-kit: Specification-driven development. github.com/github/spec-kit
8. JetBrains (2025). How to Use a Spec-Driven Approach for Coding with AI. Junie Blog.
9. ISPE (2022). GAMP 5: A Risk-Based Approach to Compliant GxP Computerized Systems. 2nd Edition.
10. ISA (1995). ISA-88.01: Batch Control Part 1: Models and Terminology.
11. ISA (2010). ISA-95.00.01: Enterprise-Control System Integration Part 1: Models and Terminology.
12. IEC (2010). IEC 61508: Functional Safety of Electrical/Electronic/Programmable Electronic Safety-Related Systems.
13. IEC (2013). IEC 61131-3: Programmable Controllers — Part 3: Programming Languages. 3rd Edition.
14. IEC (2018). IEC 60812: Failure Modes and Effects Analysis (FMEA and FMECA). 3rd Edition.
15. McKinsey & Company (2025). The State of AI in 2025. McKinsey Global Survey.
16. Anthropic (2025). Claude Code: Common Workflows. code.claude.com/docs
17. Context Studios (2026). The Vibe Coding Hangover: Why Developers Are Returning to Engineering Rigor.

AI Contribution Disclosure

This preprint was developed through human–AI collaboration using the air-gapped pipeline described in Hunt (2026b). AI agents (Anthropic Claude, Opus 4.6 and Sonnet 4.6 model tiers) contributed implementation support including literature search, structural suggestions, and text drafting. All intellectual content — the thesis, the arguments, the methodology characterisation, the claims, and all assertions of fact — are the sole responsibility of the named author. No AI agent is listed as author.

The Quantum Swarm Heating system described in Section 3 was built through the collaboration model this paper documents, using the governance framework described in Hunt (2026b). This paper is therefore both a description of the methodology and an artefact of it.